lecture03c_AutoActivationRepression

November 20, 2024

1 Auto-activation

Example of a self-activating transcriptional system where x_1 is the mRNA and x_2 is the protein:

$$\frac{dx_1}{dt} = \alpha \frac{x_2^n}{K + x_2^n} - \beta x_1$$
$$\frac{dx_2}{dt} = \gamma x_1 - \delta x_2$$

Interesting background material and further reading on this topic can be found in Michael Elowitz's lecture notes

https://biocircuits.github.io/chapters/01_intro_to_circuit_design.html

The null-clines of this system are:

$$x_1 = \frac{\alpha}{\beta} \frac{x_2^n}{K + x_2^n}$$
$$x_1 = \frac{\delta}{\gamma} x_2$$

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

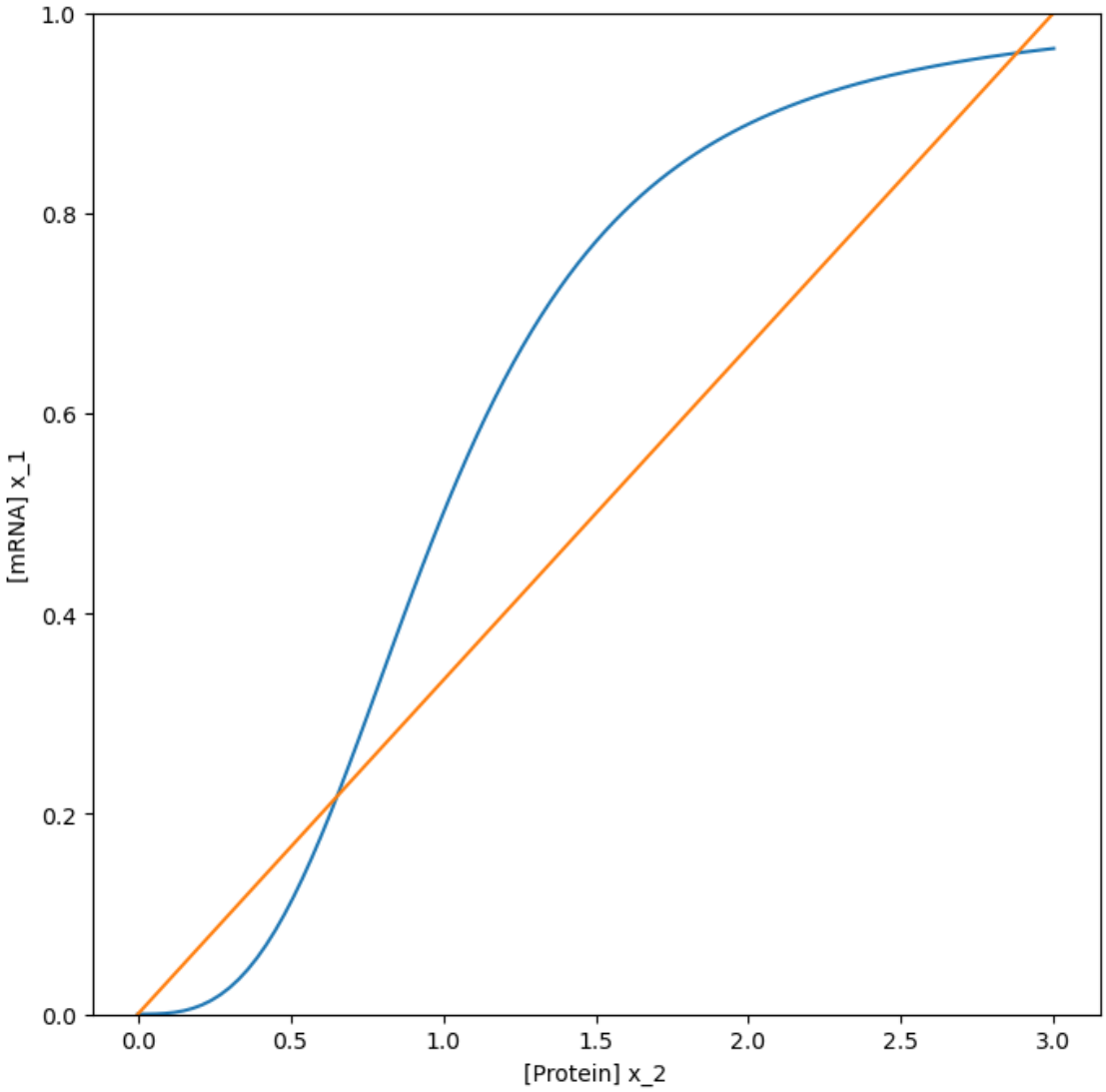
# define the function that returns the derivative
def dxdt(x,t, a,b,c,d,K,n):
    # x = [x_1, x_2]  ==> x[0] == x_1 and x[1] == x_2
    x_1, x_2 = x[0], x[1]
    dx_1 = a*x_2**n/(K+x_2**n) - b*x_1
    dx_2 = c*x_1 - d*x_2
    return [dx_1, dx_2]
```

[2]:
```
# define the parameters
b=1 # mRNA degradation rate
d=1 # protein degradation rate
a=1 # maximal mRNA production rate
```

```
c=3 # translation rate per mRNA
K=1 # half-max concentration
n=3 # cooperativity

plt.figure(figsize=(8,8))
x_2 = np.linspace(0,3,101)
plt.plot(x_2, a/b*x_2**n/(K+x_2**n))
plt.plot(x_2, d/c*x_2, c='C1')
plt.ylabel('[mRNA] x_1')
plt.xlabel('[Protein] x_2')
plt.ylim(0,1)
```

[2]: (0.0, 1.0)



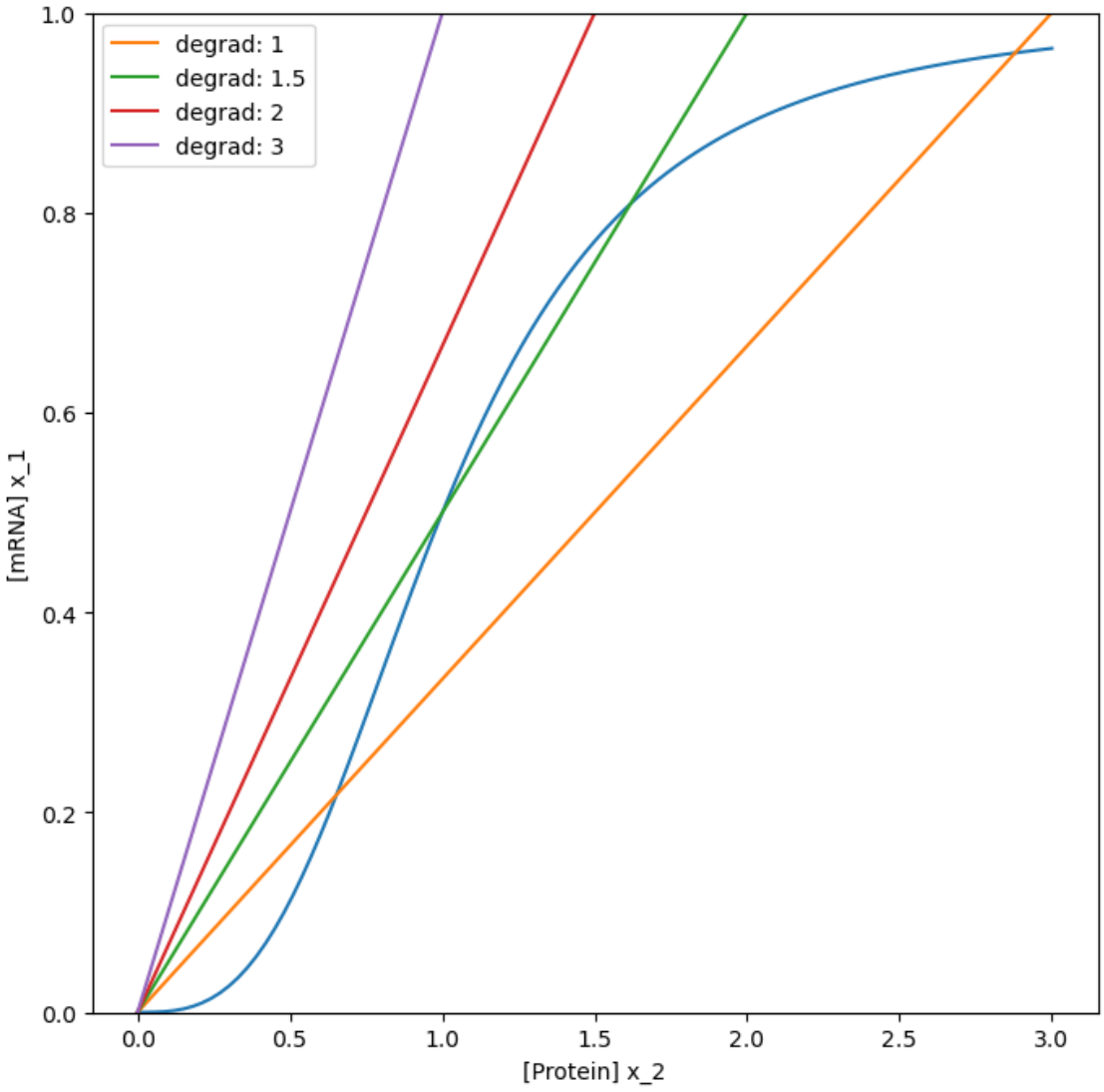
[3]:
```
plt.figure(figsize=(8,8))

x_2 = np.linspace(0,3,101)
print(x_2[:3])
plt.plot(x_2, a/b*x_2**n/(K+x_2**n))
for di, d  in enumerate([1, 1.5, 2, 3]):
    plt.plot(x_2, d/c*x_2, c=f'C{di+1}', label=f'degrad: {d}')

plt.ylabel('[mRNA] x_1')
plt.xlabel('[Protein] x_2')
plt.ylim(0,1)
plt.legend()
```

```
[0.   0.03 0.06]
```

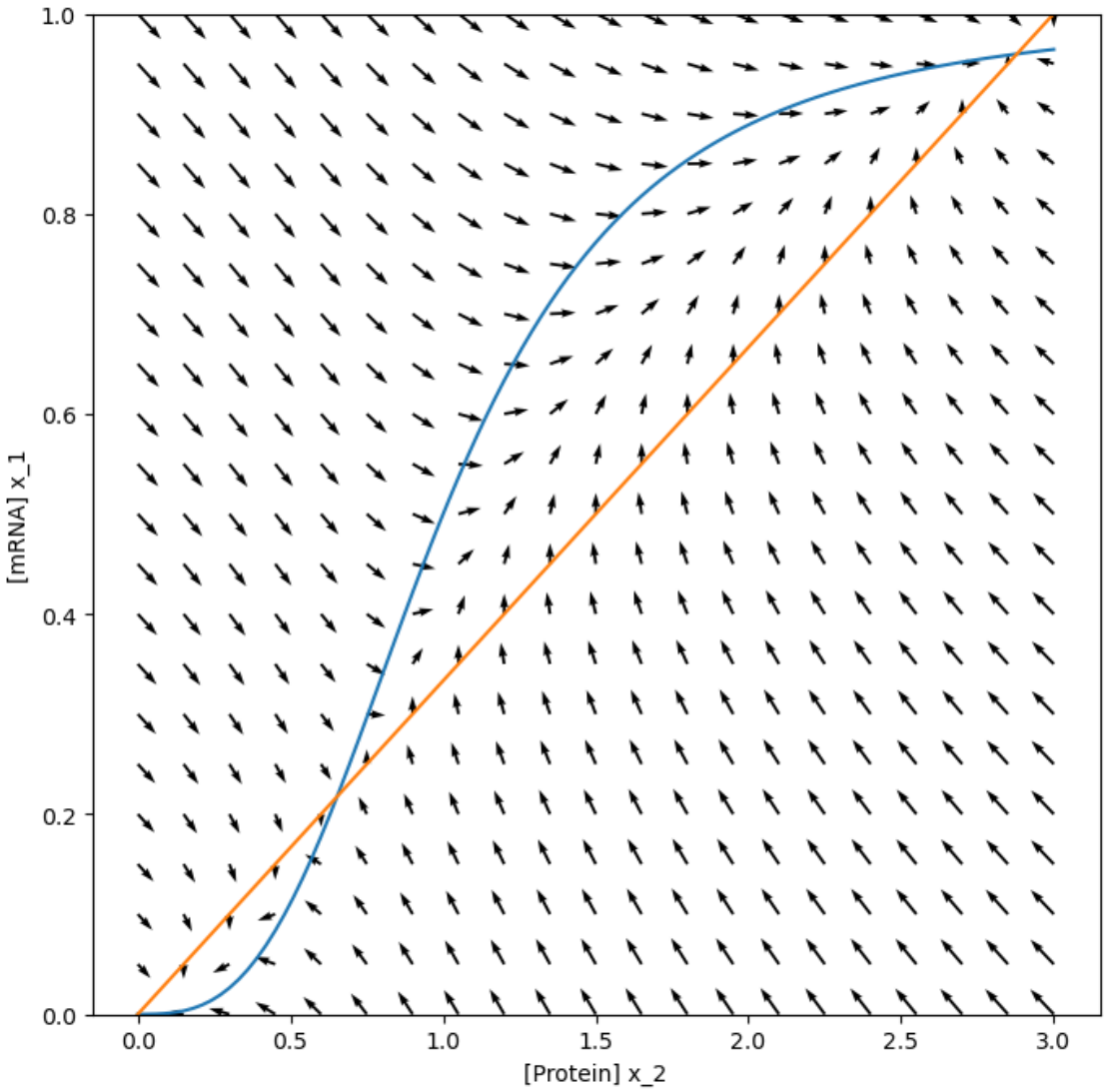
[3]:
```
<matplotlib.legend.Legend at 0x7fc090090b90>
```



[4]:
```
d = 1
plt.figure(figsize=(8,8))
x_2 = np.linspace(0,3,101)
plt.plot(x_2, a/b*x_2**n/(K+x_2**n))
plt.plot(x_2, d/c*x_2, c='C1')

plt.ylabel('[mRNA] x_1')
plt.xlabel('[Protein] x_2')
plt.ylim(0,1)
grid = np.meshgrid(np.linspace(0,1,21), np.linspace(0,3,21))
arrows = dxdt(grid,0, a,b,c,d,K,n)
arrow_length = np.sqrt(arrows[0]**2 + arrows[1]**2)+1e-10
plt.quiver(grid[1], grid[0], arrows[1]/arrow_length**0.8, arrows[0]/
 ↪arrow_length**0.8, angles='xy')
```

[4]:
```
<matplotlib.quiver.Quiver at 0x7fc090331eb0>
```



[5]:
```
d=1
plt.figure(figsize=(8,8))
plt.plot(x_2,x_2**n/(K+x_2**n), lw=4)
plt.plot(x_2, d/c*x_2, lw=4)
plt.ylabel('[mRNA] x_1')
plt.xlabel('[Protein] x_2')

t = np.linspace(0,10,101)

grid = np.meshgrid(np.linspace(0,1,31), np.linspace(0,3,31))
arrows = dxdt(grid,0, a,b,c,d,K,n)
arrow_length = np.sqrt(arrows[1]**2 + arrows[0]**2)+1e-10
```

```
plt.quiver(grid[1], grid[0], arrows[1]/arrow_length**0.8, arrows[0]/
 ↪arrow_length**0.8, angles='xy')

x0 = [0.4, 2.0]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
plt.scatter([x0[1]], [x0[0]], c="C2")
plt.plot(sol[:,1], sol[:,0], c="C2")

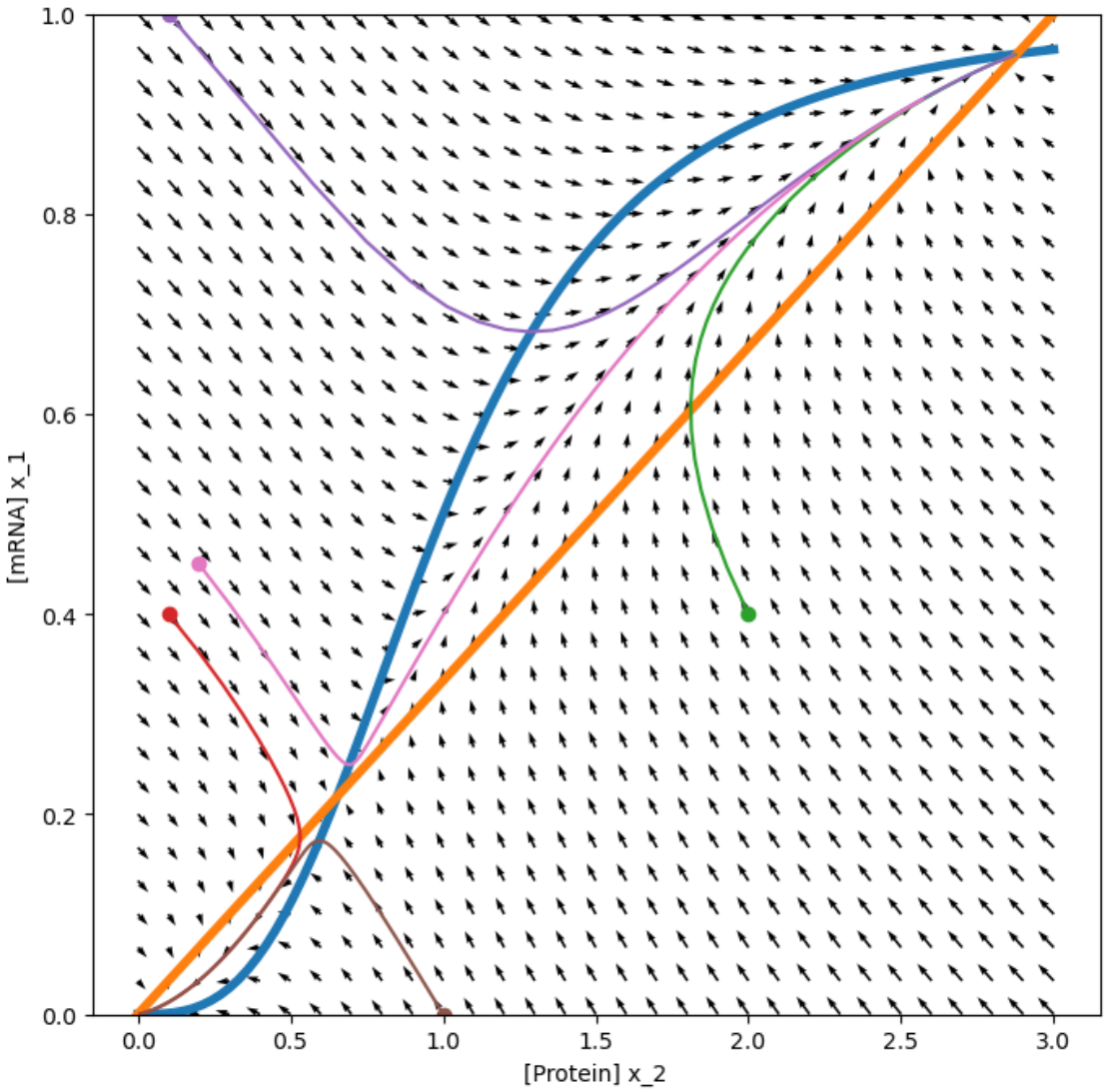

x0 = [0.4, 0.1]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
plt.scatter([x0[1]], [x0[0]], c="C3")
plt.plot(sol[:,1], sol[:,0], c="C3")

x0 = [1.0, 0.1]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
plt.scatter([x0[1]], [x0[0]], c="C4")
plt.plot(sol[:,1], sol[:,0], c="C4")


x0 = [0.0, 1.0]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
plt.scatter([x0[1]], [x0[0]], c="C5")
plt.plot(sol[:,1], sol[:,0], c="C5")


x0 = [0.45, 0.2]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
plt.scatter([x0[1]], [x0[0]], c="C6")
plt.plot(sol[:,1], sol[:,0], c="C6")
plt.ylim(0,1)
```

[5]: (0.0, 1.0)



The above example is a case of self-activation: * the protein at concentration x_2 increases transcription * mRNA x_1 translates into protein x_2 * both quantities decay

This behavior results in three fixed points: * A stable fixed point at (0,0): no protein, no transcription and no mRNA, no translation * A stable fixed point at $\approx (1, 2.9)$: degradation and production balance each other for both mRNA and protein * An unstable fixed point around $(0.2, 0.6)$: marks the point at which the system either goes to (0,0) or the high fixed point.

1.1 Self-repression

Next, we will discuss another useful example of gene regulation: auto-repression. The system of ODEs for this case is:

Example of a self-activating transcriptional system where x_1 is the mRNA and x_2 is the protein:

$$\frac{dx_1}{dt} = \alpha \frac{K}{K + x_2^n} - \beta x_1$$
$$\frac{dx_2}{dt} = \gamma x_1 - \delta x_2$$

and the corresponding null-clines are

$$x_1 = \frac{\alpha}{\beta} \frac{K}{K + x_2^n}$$
$$x_1 = \frac{\delta}{\gamma} x_2$$

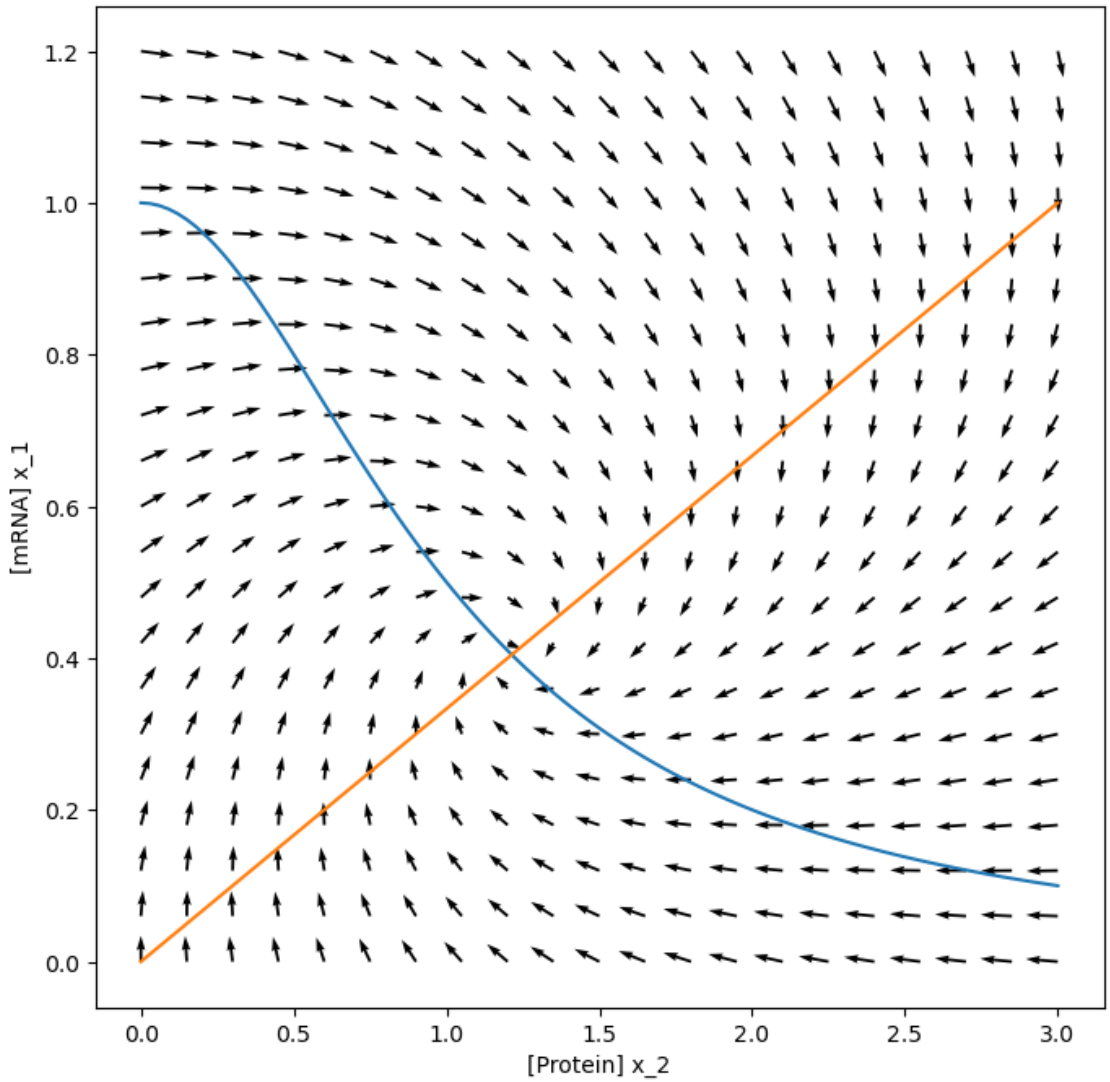
[6]:
```
# define the derivative for the auto-repressor
def dxdt(x,t, a,b,c,d,K,n):
    return [a*K/(K+x[1]**n) - b*x[0], c*x[0] - d*x[1]]

# define the parameters
b=1
d=1
a=1
c=3
K=1
n=2

# plot the null-clines
plt.figure(figsize=(8,8))
x_2 = np.linspace(0,3,101)
plt.plot(x_2, a/b*K/(K+x_2**n))
plt.plot(x_2, d/c*x_2)
plt.ylabel('[mRNA] x_1')
plt.xlabel('[Protein] x_2')

grid = np.meshgrid(np.linspace(0,1.2,21), np.linspace(0,3,21))
arrows = dxdt(grid,0, a,b,c,d,K,n)
arrow_length = np.sqrt(arrows[1]**2 + arrows[0]**2)+1e-10
plt.quiver(grid[1], grid[0], arrows[1]/arrow_length**0.8, arrows[0]/
 ↪arrow_length**0.8, angles='xy')
```

[6]:
```
<matplotlib.quiver.Quiver at 0x7fc0902b5430>
```



[7]:
```
fig, axs = plt.subplots(1,2, figsize=(10,5))
axs[0].plot(x_2, a/b*K/(K+x_2**n), lw=4)
axs[0].plot(x_2, d/c*x_2, lw=4)
axs[0].set_ylabel('[mRNA] x_1')
axs[0].set_xlabel('[Protein] x_2')
t = np.linspace(0,10,101)

grid = np.meshgrid(np.linspace(0,1,31), np.linspace(0,3,31))
arrows = dxdt(grid,0, a,b,c,d,K,n)
arrow_length = np.sqrt(arrows[1]**2 + arrows[0]**2)+1e-10
axs[0].quiver(grid[1], grid[0], arrows[1]/arrow_length**0.8, arrows[0]/
 ↪arrow_length**0.8, angles='xy')
```

```
x0 = [0.4, 2.0]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
axs[0].scatter([x0[1]], [x0[0]], c="C2")
axs[0].plot(sol[:,1], sol[:,0], c="C2")
axs[1].plot(t, sol[:,0], c="C2", label="mRNA")
axs[1].plot(t, sol[:,1], c="C2", ls='--', label="Protein")

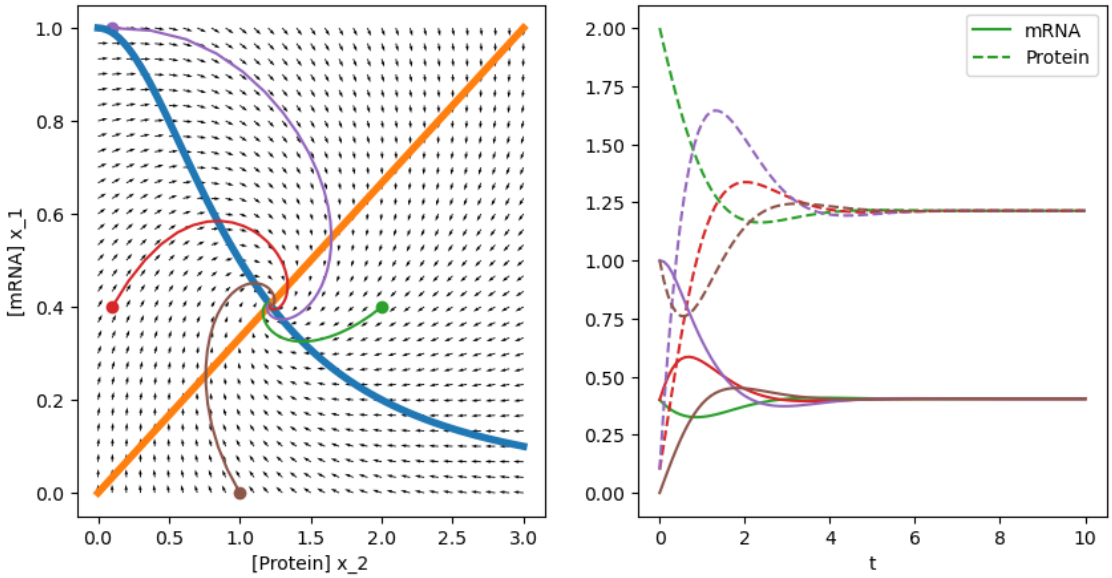

x0 = [0.4, 0.1]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
axs[0].scatter([x0[1]], [x0[0]], c="C3")
axs[0].plot(sol[:,1], sol[:,0], c="C3")
axs[1].plot(t, sol[:,0], c="C3")
axs[1].plot(t, sol[:,1], c="C3", ls='--')

x0 = [1.0, 0.1]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
axs[0].scatter([x0[1]], [x0[0]], c="C4")
axs[0].plot(sol[:,1], sol[:,0], c="C4")
axs[1].plot(t, sol[:,0], c="C4")
axs[1].plot(t, sol[:,1], c="C4", ls='--')


x0 = [0.0, 1.0]
sol = odeint(dxdt, x0, t, args=(a,b,c,d,K,n))
axs[0].scatter([x0[1]], [x0[0]], c="C5")
axs[0].plot(sol[:,1], sol[:,0], c="C5")
axs[1].plot(t, sol[:,0], c="C5")
axs[1].plot(t, sol[:,1], c="C5", ls='--')

axs[1].set_xlabel('t')
axs[1].legend()
```

[7]: <matplotlib.legend.Legend at 0x7fc08752cbc0>



1.1.1 Dig deeper

- Chance the cooperativity parameter n of the repressor (range 1 to 20). How does the behavior of the system change?
- Solve the system with the forward Euler approach that we used in the beginning of the course? (You should see the same behavior)
- Explore how stochasticity chances the dynamics of the auto-repressor! (use the forward Euler way of solving it and implement stochastic increments)

[ ]:

[ ]:

[ ]: